BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO CUỐI KỲ MÔN HỌC

HỆ THỐNG NHÚNG

STOP MODE - LOW POWER

MÃ MÔN HỌC: EMSY435664

SÁNG THỨ 6, ĐỢT 2, HỌC KÌ 1, NĂM HỌC 2025-2026

Thực hiện:

Nguyễn Ngọc Trường Phi	**23119182**
Võ Trung Nguyên	**23119180**
Hà Quang Huy	**23119147**
Vũ Minh Quang	**23119198**

Giảng viên hướng dẫn: ThS. Huỳnh Hoàng Hà

Thành phố Hồ Chí Minh, Tháng 1 năm 2026

MỤC LỤC

MỤC LỤC ..... **i**
DANH MỤC HÌNH ẢNH ..... **ii**
CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG CUNG CẤP NĂNG LƯỢNG. **1**
1.1. Các miền nguồn điện áp. 1
1.2. Bộ điều áp nguồn (Voltage Regulator) và các trạng thái hoạt động 1
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **3**
2.1. Tổng quan về chế độ Stop Mode. 3
2.1.1. Khái niệm. *3*
2.1.2. Đặc điểm. *3*
2.1.3. Nguyên lý hoạt động. *3*
2.1.4. Bảng so sánh sự khác biệt giữa các chế độ trong Low-Power. *4*
2.2. Kiến trúc điều khiển năng lượng tại lõi Cortex-M3. 4
2.3. Các cơ chế tiết kiệm năng lượng của Stop Mode. 4
2.4. Cơ chế thực thi lệnh vào Stop Mode. 4
2.5. Các nguồn đánh thức Stop Mode. 5
2.6. Cơ chế khôi phục và tiếp tục hoạt động sau Stop Mode. 5
CHƯƠNG 3: THIẾT KẾ CHƯƠNG TRÌNH THỰC NGHIỆM CHẾ ĐỘ STOP. **6**
3.1. Sơ đồ thiết kế phần cứng. 6
3.2. Flow chart. 7
3.3. Cấu hình Cubemx 10
3.4. Phân tích chương trình. 13
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH. **26**
4.1. Kết quả phần cứng. 26
4.2. Kết quả đo lường tiêu thụ dòng điện của chế độ Stop 26
4.3. Phân tích kết quả đo lường. 28
CHƯƠNG 5: KẾT LUẬN VÀ ĐÁNH GIÁ. **29**
5.1. Kết luận giá trị dòng điện tiêu thụ. 29
5.2. Đánh giá hiệu quả của chế độ Stop. 29
5.3. Ứng dụng thực tế của chế độ Stop. 29
TÀI LIỆU THAM KHẢO **30**

DANH MỤC HÌNH ẢNH.

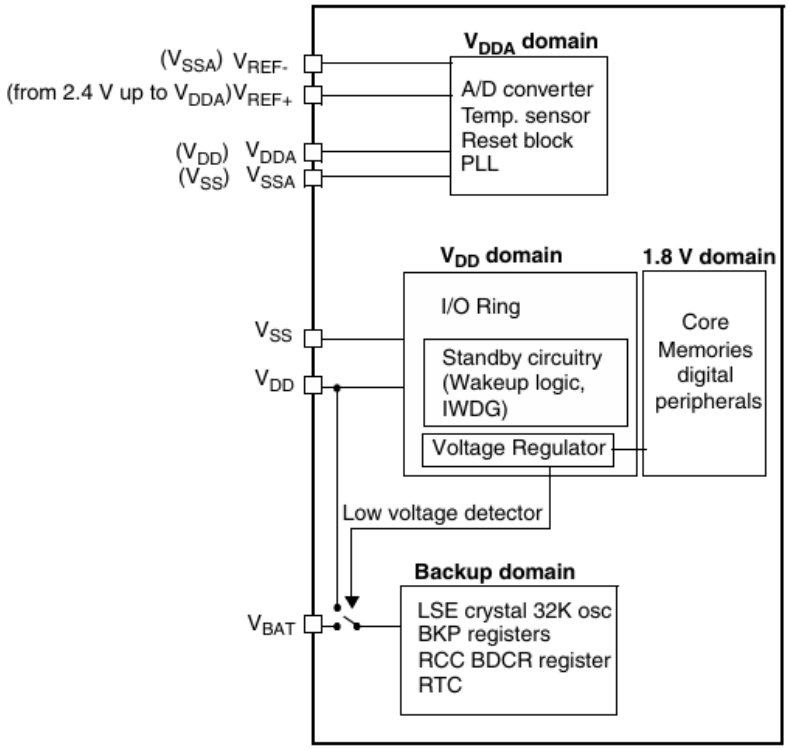
Hình 1: Tổng quan về nguồn điện.....1
Hình 2: Sơ đồ kết nối phần cứng. 6
Hình 3: Flowchart của Main loop 7
Hình 4: Flow chart của Interrupt.....8
Hình 5: Flow chart của Blinkled. 9
Hình 6 - Cấu hình chân cho vi điều khiển 11
Hình 7 - Cấu hình thạch anh cho RCC 12
Hình 8: Cấu hình NVIC 12
Hình 9: Cấu hình Xung clock. 13
Hình 10: Mạch sử dụng STM32 cho chế độ Low-power 26
Hình 11: Kết quả chạy mạch trước khi vào Stop Mode 27
Hình 12: Kết quả chạy mạch khi vào Stop Mode.....27
Hình 13: Kết quả chạy mạch khi Wake-up khỏi Stop Mode.....28

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG CUNG CẤP NĂNG LƯỢNG.

1.1. Các miền nguồn điện áp.

Vi điều khiển STM32F103C8T6 sử dụng kiến trúc lõi ARM Cortex-M3, được thiết kế với nhiều miền nguồn điện áp nhằm tối ưu hóa hiệu năng và giảm tiêu thụ năng lượng trong các chế độ Low Power, đặc biệt là Stop Mode. Các miền nguồn chính bao gồm:

- Hệ thống nguồn của STM32F103C8T6 được thiết kế chia tầng để tối ưu hóa công suất tiêu thụ:



Hình 1: Tổng quan về nguồn điện

- Miền nguồn chính (V_{DD}): Có dải điện áp từ 2.0V đến 3.6V, cấp nguồn trực tiếp cho các chân I/O và bộ điều áp nguồn nội bộ.
- Miền nguồn Analog (V_{DDA}): Có giá trị từ 2.4V đến 3.6V. Để cấp cho bộ ADC (tối thiểu 2.4V), các bộ dao động RC và PLL.
- Miền 1.8V: Được cấp bởi bộ điều áp, cung cấp năng lượng cho lõi Cortex-M3, bộ nhớ và các ngoại vi kỹ thuật số.
- Miền dự phòng (V_{BAT}): Có dải từ 1.8V đến 3.6V, dùng để duy trì bộ RTC và các thanh ghi dự phòng (BKP) khi nguồn V_{DD} chính bị ngắt.

1.2. Bộ điều áp nguồn (Voltage Regulator) và các trạng thái hoạt động

Bộ điều chỉnh điện áp luôn được kích hoạt sau khi reset. Trong STM32F103C8T6 nó đóng vai trò quan trọng trong việc kiểm soát năng lượng tại miền 1.8V thông qua ba chế độ hoạt động sau:

- Trong Run mode: Bộ điều áp cung cấp đầy đủ năng lượng để các ngoại vi hoạt động ở công suất cao nhất.

- Trong Stop mode: Bộ điều chỉnh cung cấp điện năng thấp cho miền 1,8 V, giữ nguyên nội dung của các thanh ghi và SRAM.
- Trong Standby mode: Bộ điều áp bị ngắt hoàn toàn. Đầu ra của nó chuyển sang trạng thái trở kháng cao, làm cho miền 1.8V không còn điện năng và dẫn đến mức tiêu thụ dòng điện cực thấp (chỉ khoảng 2 μA ở 3.3V).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về chế độ Stop Mode.

2.1.1. Khái niệm.

Stop Mode là chế độ tiết kiệm năng lượng của vi điều khiển STMF103C8T6, trong đó xung clock của lõi CPU và hệ thống bị ngắt, trong khi bộ điều áp nguồn và bộ nhớ SRAM vẫn được duy trì. Chế độ này giúp giảm đáng kể mức tiêu thụ điện năng nhưng vẫn cho phép hệ thống đánh thức nhanh và tiếp tục thực thi chương trình mà không cần khởi động lại.

2.1.2. Đặc điểm.

Các đặc điểm chính của Stop Mode:

- Trình ngắt hoàn toàn xung Clock CPU và hệ thống, giúp giảm mạnh công suất tiêu thụ.
- Duy trì nguồn VDD và bộ nhớ SRAM, đảm bảo dữ liệu và trạng thái hệ thống không bị mất.
- Bộ điều áp nguồn nội hoạt động ở chế độ Low-power, khác với Standby Mode khi bộ điều áp bị tắt hoàn toàn.
- Không xảy ra Reset hệ thống khi thoát Stop Mode, chương trình tiếp tục thực thi từ vị trí trước khi vào chế độ tiết kiệm năng lượng.
- Thời gian đánh thức ngắn, nhanh hơn Standby Mode.
- Có thể đánh thức thông qua hệ thống ngắt ngoài, RTC, watchdog hoặc các sự kiện từ ngoại vi cho phép.

Nhờ các đặc điểm này, Stop Mode tạo ra sự cân bằng hiệu quả giữa mức tiêu thụ năng lượng thấp và khả năng phản hồi nhanh của hệ thống.

2.1.3. Nguyên lý hoạt động.

a. Chuẩn bị vào Stop Mode

Trước khi vào Stop Mode, cần cấu hình các ngoại vi cần thiết, cho phép các nguồn đánh thức và đảm bảo không có tác vụ quan trọng thực thi.

b. Thực thi lệnh vào Stop Mode

Vi điều khiển thực hiện lệnh đặc biệt (thông quan thanh ghi quản lí nguồn và lệnh WFI hoặc WFE), yêu cầu hệ thống chuyển sang Stop Mode.

c. Ngắt xung và giảm tiêu thụ năng lượng

Sau khi vào Stop Mode, xung clock của lõi CPU và các Clock hệ thống bị ngắt. Bộ điều áp nguồn nội chuyển sang chế độ tiết kiệm năng lượng, trong khi SRAM và các thanh ghi hệ thống vẫn được duy trì.

d. Đánh thức hệ thống

Khi xảy ra một sự kiên đánh thức hợp lệ (ngắt ngoài hoặc ngắt RTC), hệ thống được kích hoạt trở lại. Clock hệ thống được khôi phục và chương trình tiếp tục thực thi từ vị trí trước khi vào Stop Mode.

2.1.4. Bảng so sánh sự khác biệt giữa các chế độ trong Low-Power.

Tiêu chí	**Sleep mode**	**Stop mode**	**Standby mode**
Xung CPU	Hoạt động	Ngắt	Tắt
SRAM	Giữ	Giữ	Mất
Bộ điều áp	Bình thường	Low-power	Tắt
Trạng thái hệ thống	Duy trì	Duy trì	Reset
Thời gian đánh thức	Rất nhanh	Nhanh	Chậm
Mức tiêu thụ năng lượng	Thấp	Rất thấp	Cực thấp
Ứng dụng điển hình	Tạm dừng ngắn	Chờ tiết kiệm năng lượng	Ngủ sâu dài hạn

2.2. Kiến trúc điều khiển năng lượng tại lõi Cortex-M3.

Các chế độ trong lowpower được xây dựng từ tính năng deep sleep của kiến trúc Cortex-M3.

Lõi Cortex-M3 hỗ trợ hai trạng thái chính liên quan đến tiết kiệm năng lượng: Sleep và Deep Sleep. Trong đó, Stop Mode của STM32F103C8T6 được hiện thực hóa từ trạng thái Deep Sleep, kết hợp với cơ chế điều khiển nguồn và clock của vi điều khiển.

Khi vào Stop Mode, bit sleepdeep trong thanh ghi System Control Register (SCR) của Cortex-M3 được thiết lập, cho phép lõi CPU ngừng hoạt động và chuyển quyền kiểm soát năng lượng cho khối quản lý nguồn của vi điều khiển. Điều này giúp giảm đáng kể mức tiêu thụ điện năng trong khi vẫn duy trì trạng thái hệ thống cần thiết.

2.3. Các cơ chế tiết kiệm năng lượng của Stop Mode.

Ngắt toàn bộ xung Clock hệ thống.

Flash memory đưa về chế độ power-down.

Bộ điều áp nguồn hoạt động ở hai mức:

- Stop Mode với Regulator hoạt động bình thường, chp phép đánh thức nhanh.
- Stop Mode với Regulator ở chế độ low-power, giúp giảm dòng tiêu thụ sâu và tăng thời gian khôi phục hệ thống.

2.4. Cơ chế thực thi lệnh vào Stop Mode.

Để đưa hệ thống vào Stop Mode, vi điều khiển STM32F103C8T6 thực hiện các bước chính sau:

- Cấu hình khối quản lí nguồn (PWR), lựa chọn chế độ Stop Mode và các tùy chọn liên quan.

- Thiết lập bit SleepDeep trong thanh ghi SCR của lõi Cortex-M3 cho phép chế độ SleepDeep.
- Thực thi WFI hoặc WFE, yêu cầu CPU ngừng hoạt động và chuyển sang Stop Mode.

Sau khi thực hiện các lệnh, hệ thống tự động ngắt xung clock CPU và hệ thống, giảm mức tiêu thụ năng lượng.

2.5. Các nguồn đánh thức Stop Mode.

Các nguồn đánh thức trong Stop Mode:

- Ngắt ngoài (EXTI) từ các chân GPIO cấu hình trước.
- Ngắt ngoài RTC, bao gồm RTC Alarm hoặc các sự kiện liên quan đến RTC.
- Ngắt nội bộ từ các ngoại vi vẫn còn được cấp nguồn trong Stop Mode.
- Sự kiện (Event) khi hệ thống sử dụng WFE.

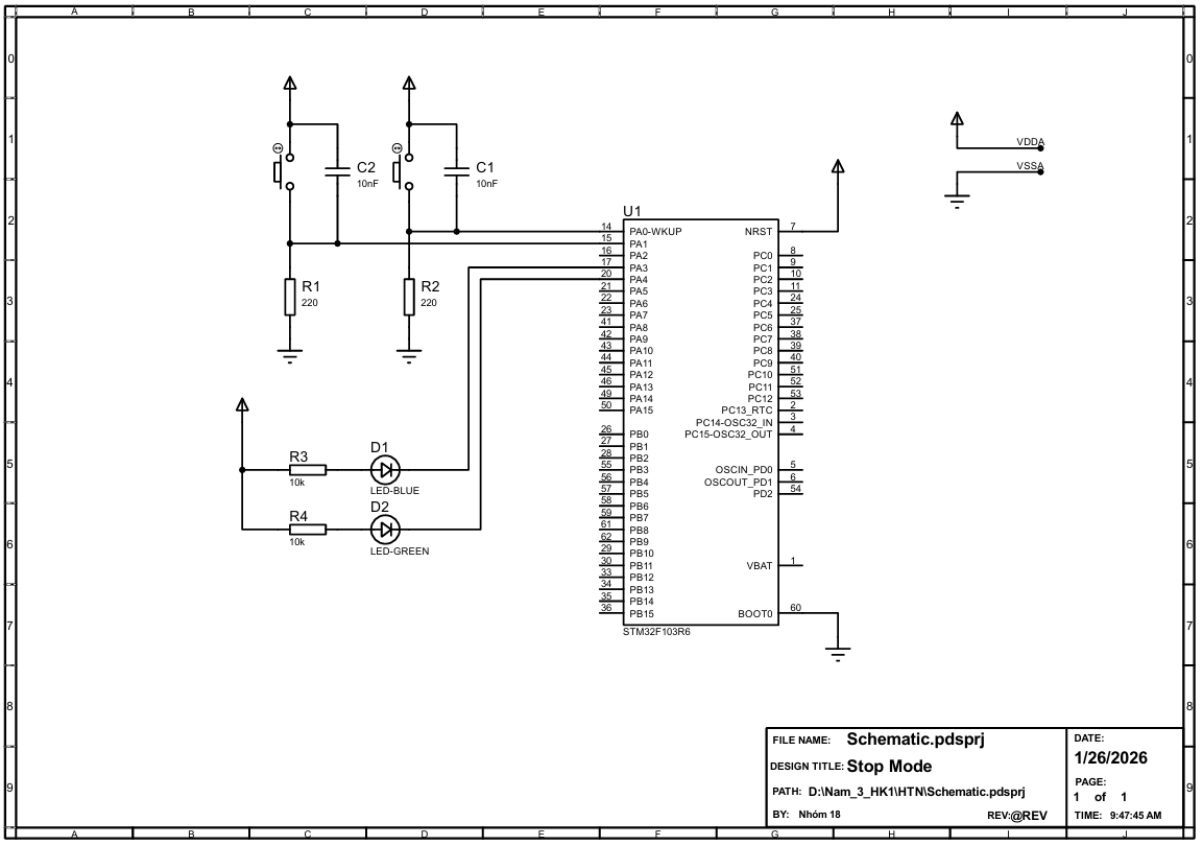
2.6. Cơ chế khôi phục và tiếp tục hoạt động sau Stop Mode.

Khi xuất hiện nguồn đánh thức hợp lệ, vi điều khiển sẽ thoát khỏi Stop Mode mà không xảy ra Reset hệ thống. CPU Cortex-M3 tiếp tục thực thi chương trình tại vị trí ngay sau lệnh WFI/WFE.

Xung clock hệ thống đã bị dừng trong Stop Mode, chương trình cần cấu hình lại hệ thống Clock (HSI/HSE và PLL) để đưa vi điều khiển trở về chế độ hạot động bình thường. Dữ liệu trong SRAM và các thanh ghi vẫn được duy trì, hệ thống tiếp tục hoạt động với thời gian trế thấp.

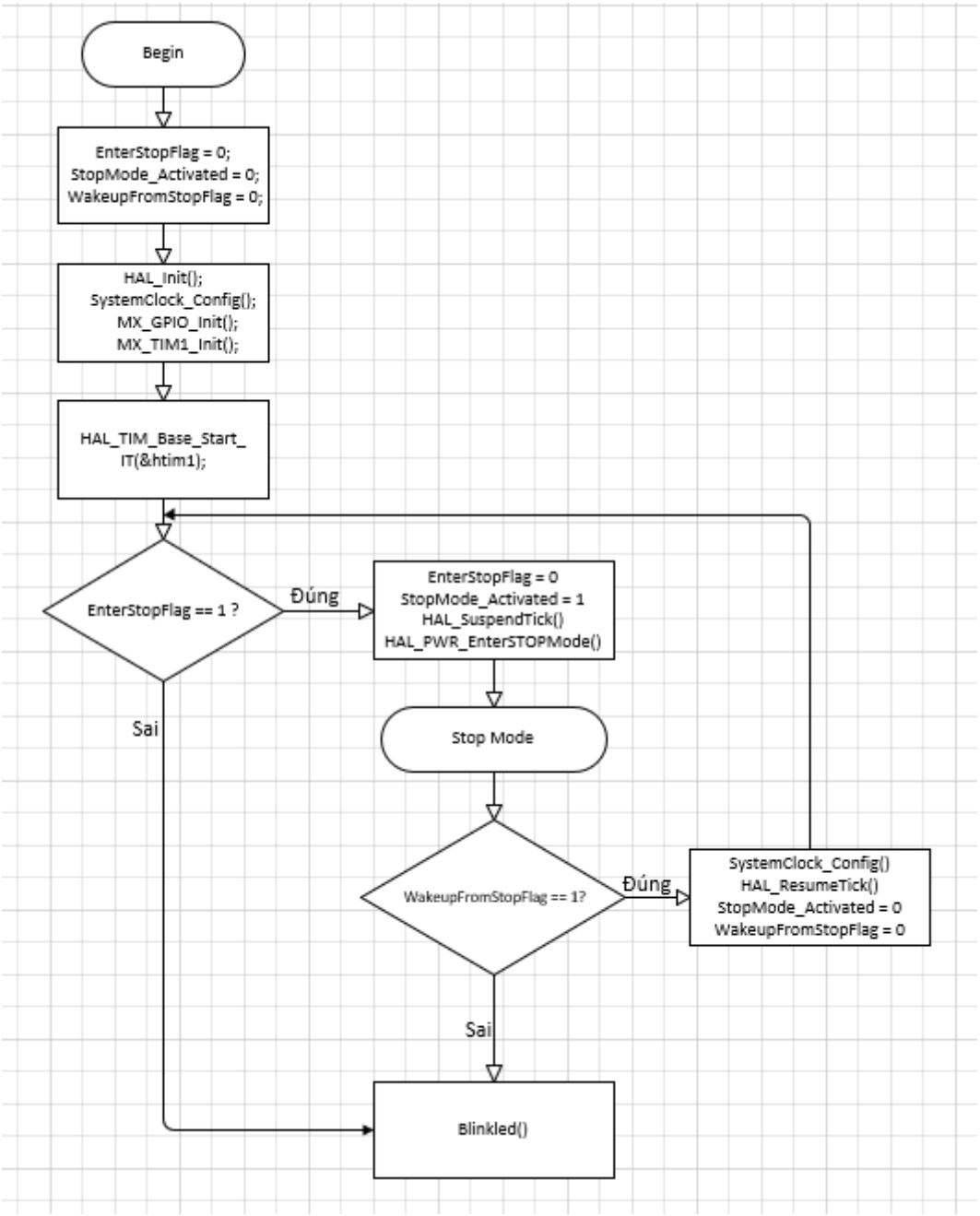
CHƯƠNG 3: THIẾT KẾ CHƯƠNG TRÌNH THỰC NGHIỆM CHẾ ĐỘ STOP.

3.1. Sơ đồ thiết kế phần cứng.



Hình 2: Sơ đồ kết nối phần cứng.

3.2. Flow chart.



Hình 3: Flowchart của Main loop

Phân tích:

1. Khởi tạo hệ thống
   - Chương trình đặt cờ cho chế độ Stop, Wake-up và chế độ vào Stop là 0.
   - Tiến hành khởi động các thành phần cơ bản: thư viện HAL, xung Clock, GPIO, bộ định thời Tim1 và bộ ngắt.
2. Kiểm tra vào chế độ Stop

- Khi EnterStopFlag = 1, cờ StopFlag sẽ tắt và chế độ Stop = 1 báo hiệu đang trong Stop mode.

  + SuspendTick tắt để không bị đánh thức sau 1s.

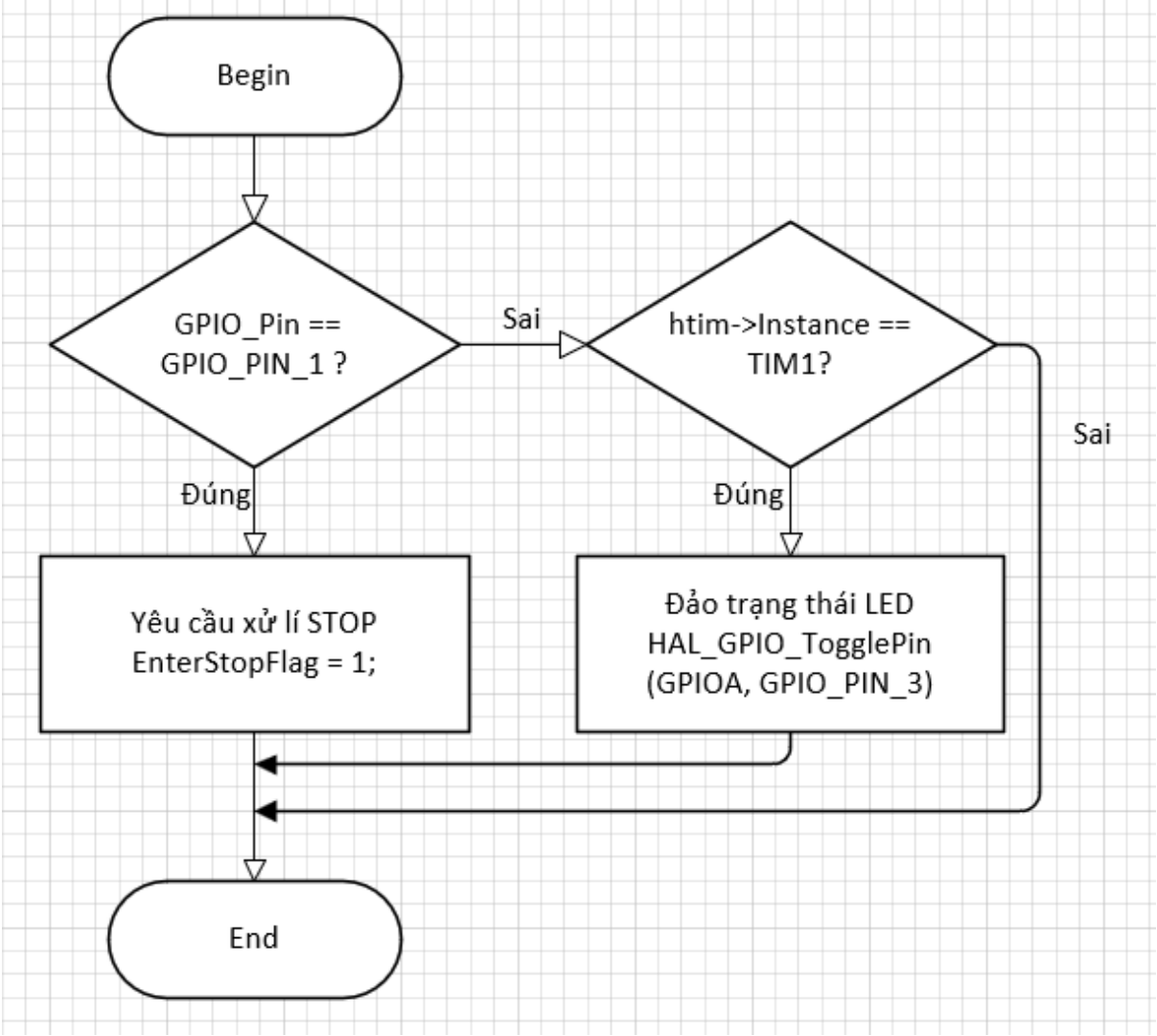
  + Vào chế độ Stop khi PA1 nhấn.

  + Kiểm tra chế độ đánh thức, khi WakeupFromStop = 1, quay về trạng thái kiểm tra EnterStopFlag.

  + Khi WakeupFromStop = 0, led nhấp nháy.

- Khi EnterStopFlag = 0, led tiếp tục nhấp nháy.

3. Gọi hàm blinkled(): hàm được gọi khi đã xảy ra quá trình Wake-up và có Stop = 0.



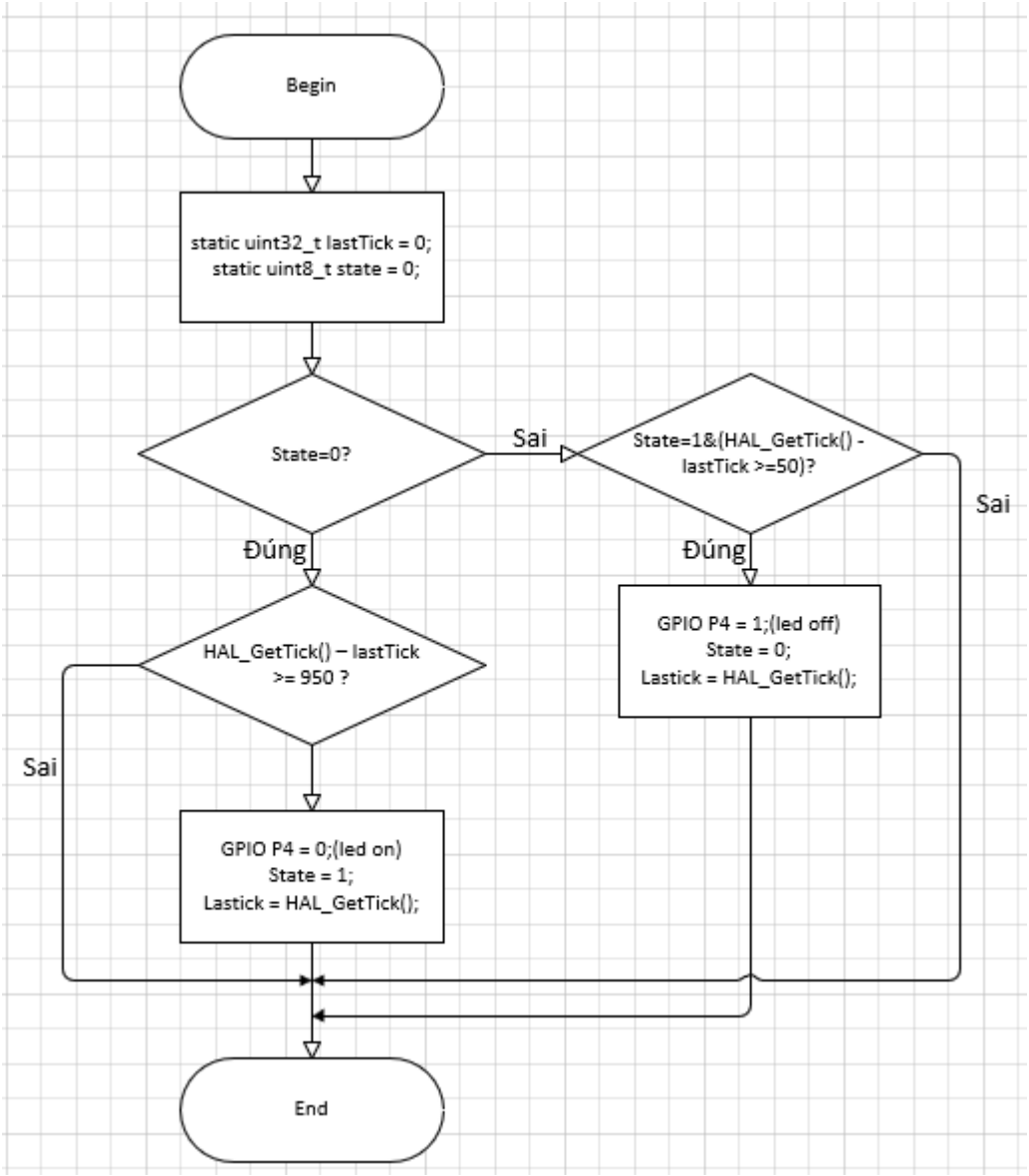
Hình 4: Flow chart của Interrupt.

Phân tích:

1. Kiểm tra ngắt từ GPIO

- Kiểm tra điều kiện khi GPIO_Pin == GPIO_PIN1
  + Nếu đúng ngắt được kích hoạt từ nút nhấn PA1, khi đó chương trình không xử lí Stop mà chỉ thiết lập cờ.

2. Kiểm tra gắt tử Timer
   - Nếu htim->Instance == TIM1 đúng, ngắt được xác định là từ bộ định thời TIM1.
   - Khi đó, chương trình thực hiện đảo trạng thái LED thông qua lệnh: HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_3);, choh phép led nhấp nháy theo chu kỳ timer1.
3. Nếu không thỏa điều kiện.
   - Chương trình không thực hiện hành động nào và kết thúc quá trình xử lý ngắt hiện tại.



Hình 5: Flow chart của Blinkled.

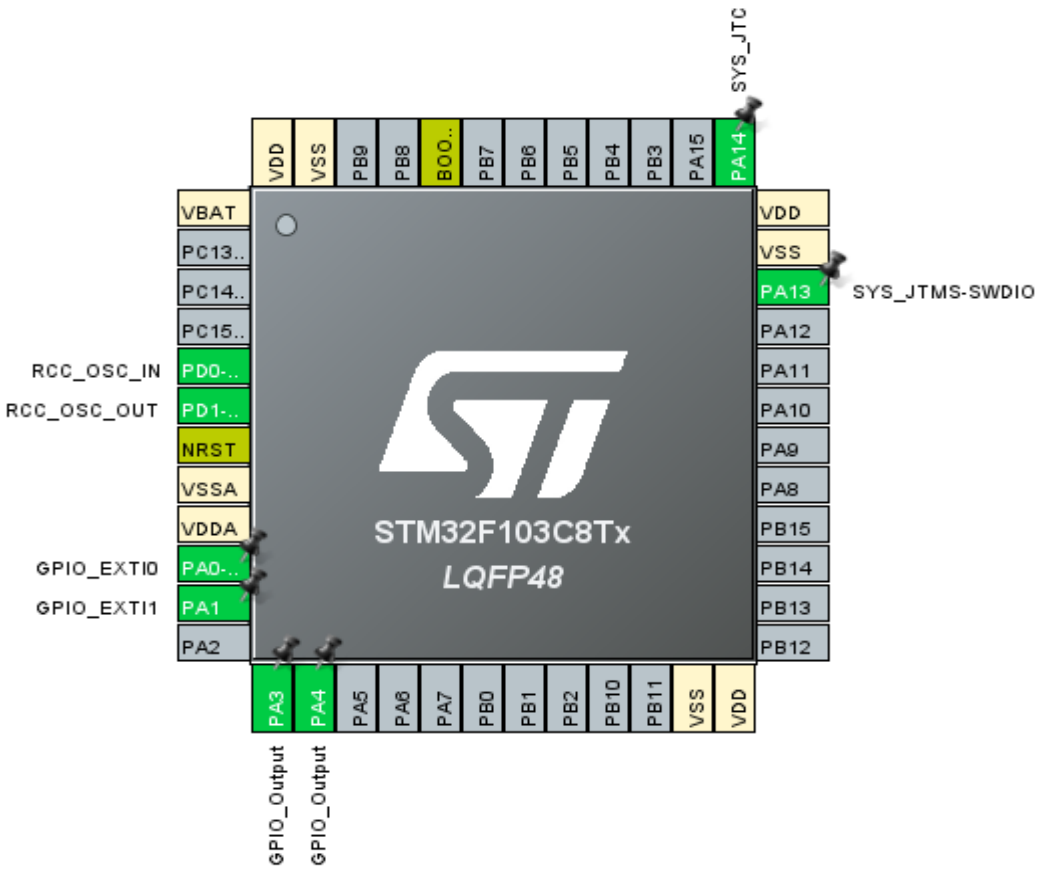
Phân tích:

1. Khởi tạo hệ thống
   - Tại thời điểm bắt đầu, chương trình khai báo hai biến tĩnh:
     + lastTick: dùng để lưu mốc thời gian gần nhất khi trạng thái LED thay đổi.
     + state: dùng để lưu trạng thái hiện tại của LED.

2. Trạng thái State
   - Khi state=0, hệ thống led ở trạng thái tắt, chương trình kiểm tra điều kiện thời gian
   - Nếu điều kiện thỏa mãn, chân PA4 bật và lastTick chuyển sang cập nhật thời gian.
   - Nếu không thỏa mãn, chương trình không hành động và quay trở lại vòng lặp.
3. Trạng thái State = 1.
   - Khi state=1, hệ thống led đang bật, chương trình kiểmt tra điều kiện thời gian:
     + Nếu chưa thỏa mãn, led vẫn giữ trạng thái bật.
     + Nếu thỏa mãn, led tắt, state về 0 và lastTick cập nhật.
4. Chu trình hoạt động
   - Hai trạng thái led được cập nhật liên tục trong vòng lặp chính, nhấp nháy theo chu kỳ: led tắt trong 950ms và bật trong 50ms.
   - Hàm delay() không làm gián đoạn chương trình chính, thực hiện tác vụ song song.

3.3. Cấu hình Cubemx

a. Cấu hình GPIO



Hình 6 - Cấu hình chân cho vi điều khiển

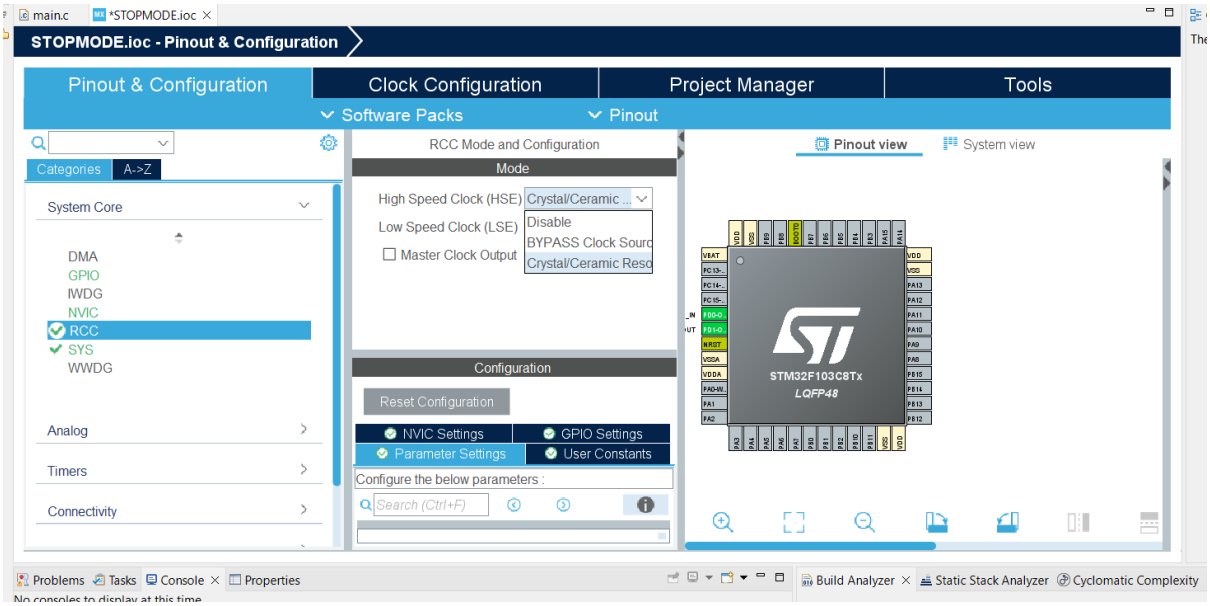
Chân PA0: chân Interrupt ngắt ngoài EXTI0.

Chân PA1: chân Interrupt ngắt ngoài EXTI1.

Chân PA3: chân GPIO_OUTPUT.

Chân PA4: chân GPIO_OUTPUT.

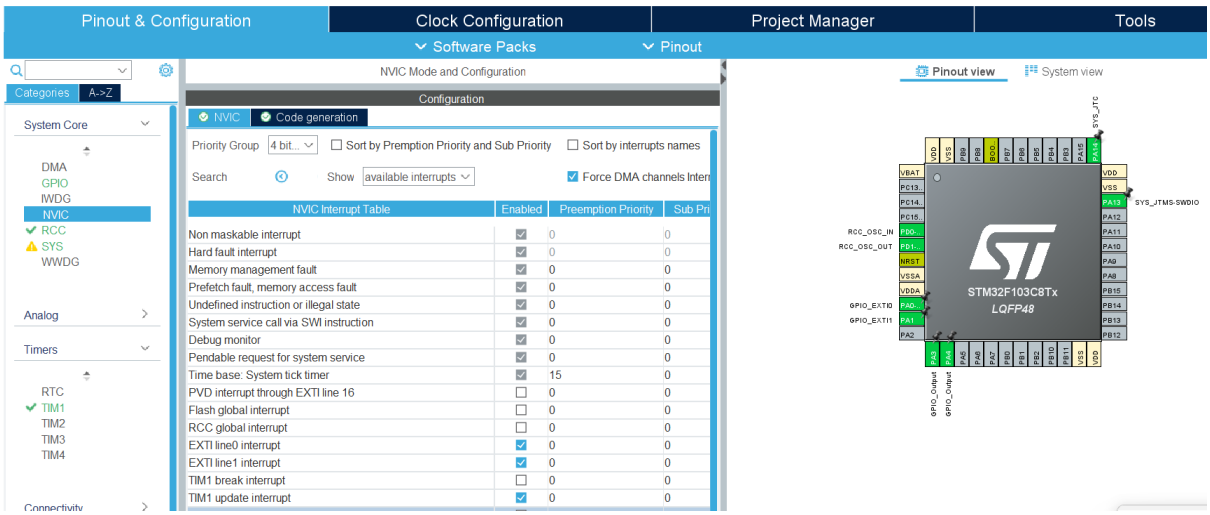
b. Cấu hình cho RCC



Hình 7 - Cấu hình thạch anh cho RCC

- Cấu hình thạch anh cho HSE với tần số 8MHz.

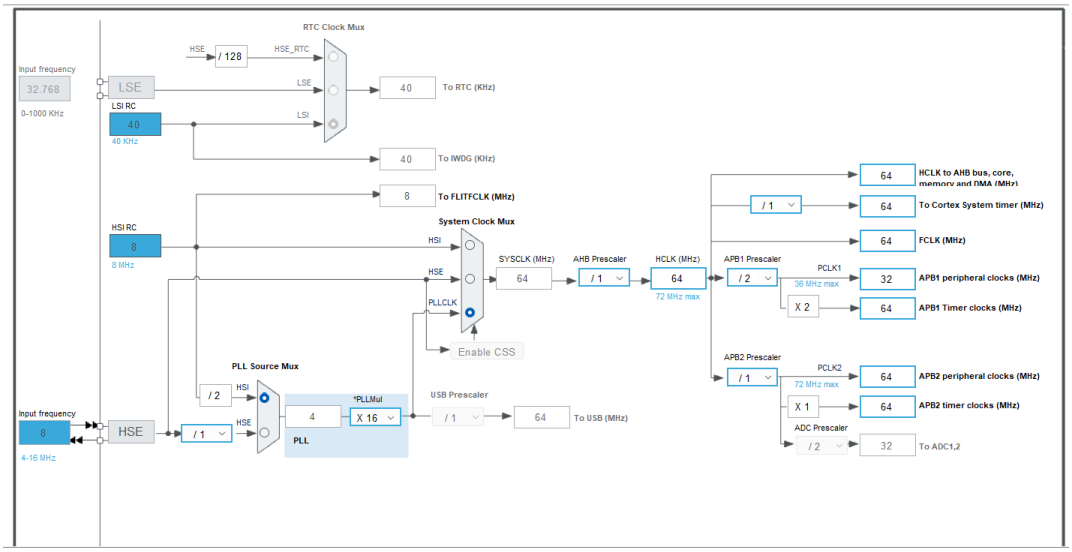
c. Cấu hình NVIC



Hình 8: Cấu hình NVIC

- Cho phép EXTI0, EXTI1 thực hiện ngắt ngoài.
- Cho phép bộ định thời TIM1 cập nhật.

d. Cấu hình xung Clock



Hình 9: Cấu hình Xung clock.

Trong chế độ Stop mode, mạch vòng khóa pha (PLL) và các xung nhịp hệ thống (SYSCLK, HCLK, APB1, APB2) được tắt hoàn toàn nhằm giảm thiểu tiêu thụ năng lượng. Chỉ các nguồn xung thấp như LSI hoặc LSE có thể được duy trì để phục vụ các khối chức năng cần thiết như RTC hoặc mạch đánh thức (EXTI). Khi thoát khỏi Stop mode, hệ thống cần cấu hình lại PLL và xung clock trước khi tiếp tục hoạt động bình thường. Nhờ việc ngắt các dao động cao tần, Stop mode đạt mức tiêu thụ năng lượng thấp hơn đáng kể so với Sleep mode, trong khi vẫn giữ được nội dung SRAM và thanh ghi.

3.4. Phân tích chương trình.

Đoạn code điều khiển STM32 quản lý năng lượng với 2 chế độ tiết kiệm điện: Sleep Mode và Stop Mode, điều khiển bằng 2 nút nhấn và hiển thị 2 trạng thái Led.

Code chính

```
/* USER CODE BEGIN Header */
/**

******************************************************************************
*****
  * @file           : main.c
  * @brief          : Main program body

******************************************************************************
*****
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
```

```
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *

******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
uint8_t StopMode_Activated = 0;   // Flag: CPU is in stop
uint8_t EnterStopFlag = 0;        // Flag: request to enter stop
uint8_t WakeupFromStopFlag = 0;
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
```

```
TIM_HandleTypeDef htim1;

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM1_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void blinkled()
{
		static uint32_t lastTick = 0;
   static uint8_t state = 0;

		if(!state&&(HAL_GetTick() - lastTick >=950)){
				HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4,0);
				state = 1;
				lastTick = HAL_GetTick();
		}
		else if(state&&(HAL_GetTick() - lastTick >=50)){
				HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4,1);
				state = 0;
				lastTick = HAL_GetTick();
		}
}

// Timer interrupt for led blink
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
```

```
                if(htim->Instance == htim1.Instance){
                        HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_3);
                }
}
/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_TIM1_Init();
```

```
  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim1);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
                  // stop mode entry and wakeup
                  if (EnterStopFlag)
    {
       EnterStopFlag = 0;
       StopMode_Activated = 1;
       HAL_SuspendTick();
       HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON,
PWR_STOPENTRY_WFI);
    }
                  if (WakeupFromStopFlag)
                  {
    SystemClock_Config();    // khôi ph?c clock
    HAL_ResumeTick();        // b?t l?i SysTick

    StopMode_Activated = 0;
    WakeupFromStopFlag = 0;
                  }


                  blinkled();
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
```

```
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL8;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType                                          =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}
```

```
/**
  * @brief TIM1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM1_Init(void)
{

  /* USER CODE BEGIN TIM1_Init 0 */

  /* USER CODE END TIM1_Init 0 */

  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};

  /* USER CODE BEGIN TIM1_Init 1 */

  /* USER CODE END TIM1_Init 1 */
  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 63999;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 499;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
```

```
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM1_Init 2 */

  /* USER CODE END TIM1_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);

  /*Configure GPIO pins : PA0 PA1 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

```
  /*Configure GPIO pins : PA3 PA4 */
  GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI0_IRQn);

  HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI1_IRQn);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
/**
  * @brief EXTI line callback for PA0
  */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   if (GPIO_Pin == GPIO_PIN_0)   // PA0: Enter STOP
   {
      EnterStopFlag = 1;
   }

   if (GPIO_Pin == GPIO_PIN_1)   // PA1: Wakeup from STOP
   {
      WakeupFromStopFlag = 1;
   }
}
```

```
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Chương trình thực hiện các chức năng chính:

- Điều khiển Led nhấp nháy theo chu kỳ:
  + Led PA4 nháy dựa trên vòng lặp chính. (950ms tắt, 50ms sáng).

+ Led PA3 nháy dựa theo cơ chế ngắt ngoài.

- Vào chế độ tiết kiệm năng lượng Stop Mode sử dụng nút nhấn PA0.
- Đánh thức khỏi chế độ Stop sử dụng nút nhấn PA1.

Phân tích code:

1. Các biến toàn cục

```
/* USER CODE BEGIN PD */
uint8_t StopMode_Activated = 0;    // Flag: CPU is in stop
uint8_t EnterStopFlag = 0;         // Flag: request to enter stop
uint8_t WakeupFromStopFlag = 0;
```

Sử dụng các cờ để đồng bộ giữa ngắt và vòng lặp chính.

2. Hàm Blinkled()

```
61  void blinkled()
62  {
63      static uint32_t lastTick = 0;
64      static uint8_t state = 0;
65
66      if(!state&&(HAL_GetTick() - lastTick >=950)){
67          HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4,0);
68          state = 1;
69          lastTick = HAL_GetTick();
70      }
71      else if(state&&(HAL_GetTick() - lastTick >=50)){
72          HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4,1);
73          state = 0;
74          lastTick = HAL_GetTick();
75      }
76  }
77
```

- Chu kỳ led sáng 50ms, tắt 950 ms (chu kỳ 1s).
- Kết nối led với actiic-low (0 = sáng, 1= tắt)
- Dùng Hal_GetTick() để tạo thời gian không chặn.

3. Callback ngắt Timer

```
78  // Timer interrupt for led blink
79  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
80  {
81      if(htim->Instance == htim1.Instance){
82          HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_3);
83      }
84  }
```

- Timer ngắt sau mỗi 500ms.
- Led PA3 nhấp nháy với tần số 1Hz (500ms sáng, 500ms tắt).

4. Cấu hình Timer1

```
207   /* USER CODE END TIM1_Init 1 */
208   htim1.Instance = TIM1;
209   htim1.Init.Prescaler = 63999;
210   htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
211   htim1.Init.Period = 499;
212   htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
213   htim1.Init.RepetitionCounter = 0;
214   htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
```

- Clock hệ thống ~64MHz.
- Tần số Timer là 1ms.
- Thời gian ngắt là 500ms.

5. Vòng lặp chính

```
126     if (EnterStopFlag)
127     {
128         EnterStopFlag = 0;
129         StopMode_Activated = 1;
130         HAL_SuspendTick();
131         HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON, PWR_STOPENTRY_WFI);
132     }
133     if (WakeupFromStopFlag)
134     {
135     SystemClock_Config();    // khôi ph?c clock
136     HAL_ResumeTick();        // b?t l?i SysTick
137
138     StopMode_Activated = 0;
139     WakeupFromStopFlag = 0;
140     }
141
142
143     blinkled();
```

- Xóa cờ tránh vào Stop nhiều lần, sau đó báo trạng thái Stop.
- Chương trình dừng khi gọi lệnh HAL_PWR_EnterSTOPMode
- Chương trình thức dậy sau lệnh Wakeup dựa vào quá trình khôi phục Clock, bật SysTick
- Led quay trở lại hoạt động bình thường.

6. Ngắt ngoài (EXTI)

```c
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)    // PA0: Enter STOP
    {
        EnterStopFlag = 1;
    }

    if (GPIO_Pin == GPIO_PIN_1)    // PA1: Wakeup from STOP
    {
        WakeupFromStopFlag = 1;
    }
}
```

- Khi PA0 yêu cầu vào Stop.
- Khi PA1 yêu cầu thức dậy từ Stop.

7. Cấu hình Clock

```c
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL8;
```

- Đặt lại HSE = 8MHz, PLL = 64MHz, SYSCLK = 64MHz.

8. Xử lí lỗi hệ thống

```c
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
```

- Khi lỗi xảy ra, chương trình tắt toàn bộ ngắt và giữ vi điều khiển trong vòng lặp vô hạn.

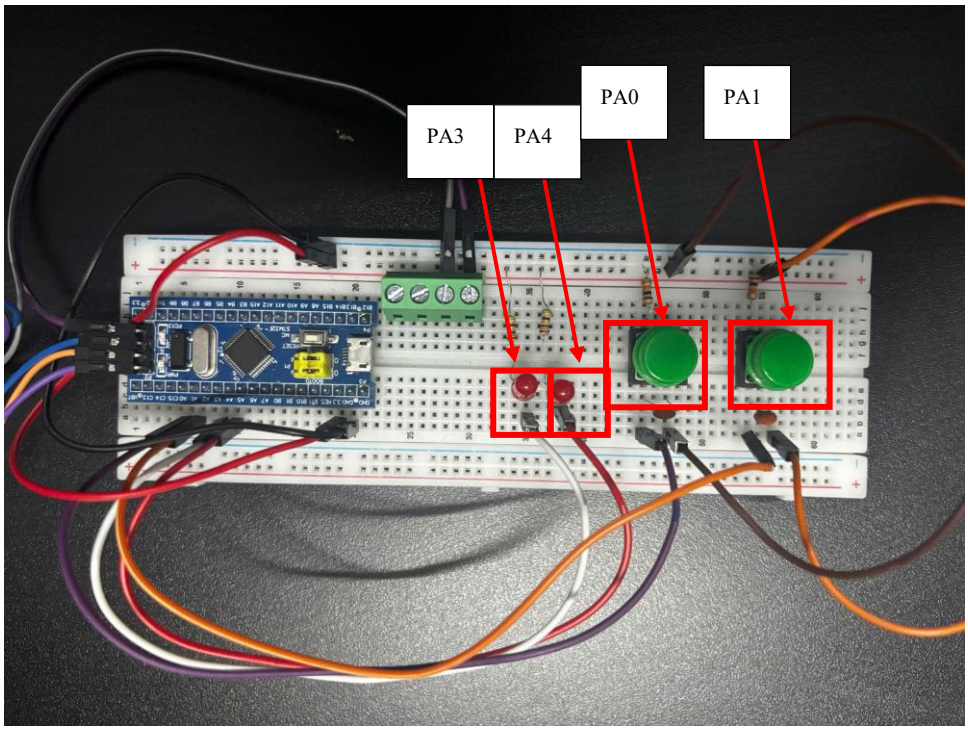
9. Khởi tạo ngoại vi

```c
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_TIM1_Init();
  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim1);
```

- PA0, PA1 sử dụng interrupt
- PA3, PA4 dùng cho output LED
- Tim1 có interrupt

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH.

4.1. Kết quả phần cứng.

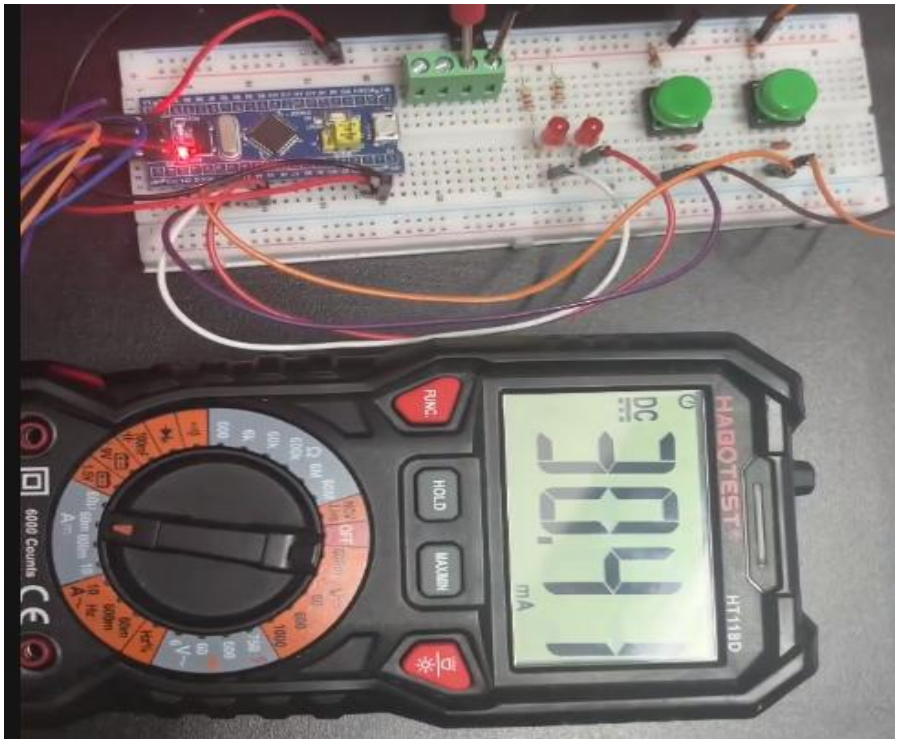


Hình 10: Mạch sử dụng STM32 cho chế độ Low-power

- Nút nhấn PA1: Thoát chế độ stop.
- Nút nhấn PA0: Vào chế độ Stop.
- Led PA3: Nhấp nháy theo cơ chế Ngắt.
- LED PA4: Nháp nháy theo hàm main loop().

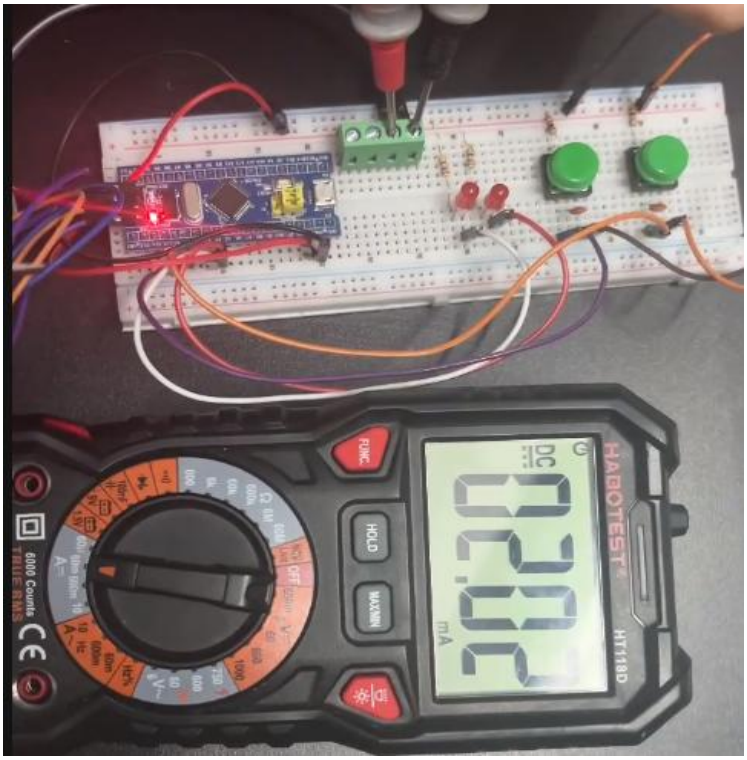
4.2. Kết quả đo lường tiêu thụ dòng điện của chế độ Stop.

a. Trước khi vào Stop Mode



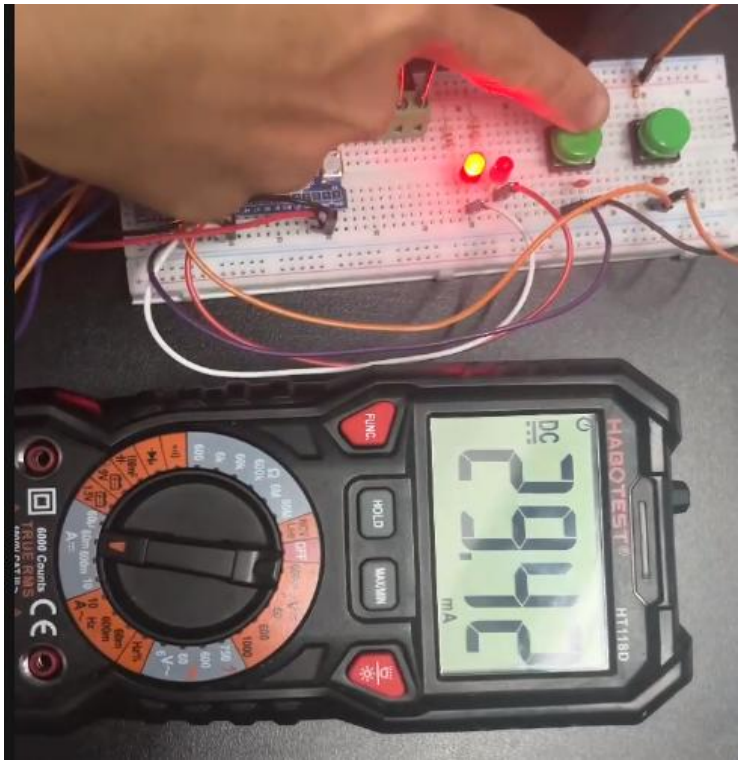
Hình 11: Kết quả chạy mạch trước khi vào Stop Mode

b. Sau khi vào Stop Mode



Hình 12: Kết quả chạy mạch khi vào Stop Mode

b. Thức dậy sau khi Stop Mode



Hình 13: Kết quả chạy mạch khi Wake-up khỏi Stop Mode

4.3. Phân tích kết quả đo lường.

- Giá trị lý thuyết theo datasheet STM32F103, ở chế độ Stop dòng tiêu thụ điện nằm trong khoảng 0.2-1mA.
- Theo giá trị thực tế đã đo được, khi vào chế độ Stop dòng điện rơi vào khoảng 0.2mA rất gần so với lý thuyết.

CHƯƠNG 5: KẾT LUẬN VÀ ĐÁNH GIÁ.

5.1. Kết luận giá trị dòng điện tiêu thụ.

Ở chế độ Stop mode**,** vi điều khiển tiêu thụ dòng điện rất thấp khoảng 2mA do CPU và hầu hết các xung clock bị dừng, trong khi dữ liệu trong SRAM vẫn được giữ. Nhờ đó, Stop mode giúp tiết kiệm năng lượng hiệu quả và kéo dài thời gian hoạt động của hệ thống. Thông số đo này đúng với kết quả đã có trên datasheet của nhà sản xuất.

5.2. Đánh giá hiệu quả của chế độ Stop.

Chế độ Stop mode cho thấy hiệu quả cao trong việc tiết kiệm năng lượng cho vi điều khiển STM32. Khi vào Stop mode, CPU và toàn bộ hệ thống xung clock bị tắt, trong khi bộ nhớ RAM và các thanh ghi vẫn được duy trì. Nhờ đó, dòng tiêu thụ của hệ thống giảm xuống mức rất thấp, chỉ còn vài chục microampere so với hàng chục milliampere ở chế độ hoạt động bình thường.

Bên cạnh khả năng tiết kiệm năng lượng, Stop mode vẫn đảm bảo chương trình không bị gián đoạn. Sau khi được đánh thức bằng ngắt ngoài, vi điều khiển không bị reset mà tiếp tục thực thi chương trình. Tuy nhiên, hệ thống cần cấu hình lại xung clock và SysTick để đảm bảo các chức năng thời gian hoạt động chính xác. Nhìn chung, Stop mode đáp ứng tốt yêu cầu tiết kiệm năng lượng và khả năng khôi phục nhanh của hệ thống nhúng.

5.3. Ứng dụng thực tế của chế độ Stop.

Chế độ Stop được ứng dụng rộng rãi trong các hệ thống nhúng yêu cầu tiết kiệm năng lượng, đặc biệt là các thiết bị sử dụng pin. Một số ứng dụng tiêu biểu cho chế này bao gồm hệ thống IoT, mạng cảm biến không dây, thiết bị đo lường môi trường và các thiết bị điều khiển thông minh. Với ưu điểm dòng tiêu thụ thấp, thời gian wake-up nhanh và không mất dữ liệu RAM, Stop mode là giải pháp phù hợp cho nhiều ứng dụng nhúng hiện nay.

TÀI LIỆU THAM KHẢO

[1] STMicroelectronics**,** *STM32F103x8 / STM32F103xB – Medium-density performance line Arm®-based 32-bit MCU with 64 or 128 Kbytes Flash memory, USB, CAN, 7 timers, 2 ADCs, 9 communication interfaces*, Datasheet, STMicroelectronics.

[2] STMicroelectronics, *UM1850 – Description of STM32F1 HAL and low-layer drivers*, STMicroelectronics.

[3] STM32 Low Power Modes Explained – Sleep, Stop, Standby & Power Consumption: *https://controllerstech.com/low-power-modes-in-stm32/***.**